Method Objects

For the sake of completeness, you might want to take one more look at the last member of the callable objects' family: methods. If you're not convinced that methods, like lambdas, are just callable objects, look at this code:

blocks/methods.rb

```
class MyClass
  def initialize(value)
    @x = value
  end
  def my_method
    @x
  end
end

object = MyClass.new(1)
m = object.method :my_method
m.call                              # => 1
```

By calling Kernel#method, you get the method itself as a Method object, which you can later execute with Method#call. In Ruby 2.1, you also have Kernel#singleton_method, which converts the name of a *Singleton Method* *(114)* to a Method object. (What are you saying? You don't know what a Singleton Method is yet? Oh, you will, you will…)

A Method object is similar to a block or a lambda. Indeed, you can convert a Method to a Proc by calling Method#to_proc, and you can convert a block to a method with define_method. However, an important difference exists between lambdas and methods: a lambda is evaluated in the scope it's defined in (it's a closure, remember?), while a Method is evaluated in the scope of its object.

Ruby has a second class that represents methods—one you might find perplexing. Let's have a look at it first, and then we'll see how it can be used.

Unbound Methods

UnboundMethods are like Methods that have been detached from their original class or module. You can turn a Method into an UnboundMethod by calling Method#unbind. You can also get an UnboundMethod directly by calling Module#instance_method, as in the following example:

blocks/unbound_methods.rb

```
module MyModule
  def my_method
    42
  end
end
```